JavaScript is disabled on your browser.

compact3
java.lang.management

Interface ThreadMXBean

All Superinterfaces:
`PlatformManagedObject`

```
public interface ThreadMXBean
extends PlatformManagedObject
```

The management interface for the thread system of the Java virtual machine.

A Java virtual machine has a single instance of the implementation class of this interface. This instance implementing this interface is an MXBean that can be obtained by calling the `ManagementFactory.getThreadMXBean()` method or from the `platform MBeanServer` method.

The `ObjectName` for uniquely identifying the MXBean for the thread system within an MBeanServer is:

```
java.lang:type=Threading
```

It can be obtained by calling the `PlatformManagedObject.getObjectName()` method.

Thread ID

Thread ID is a positive long value returned by calling the `Thread.getId()` method for a thread. The thread ID is unique during its lifetime. When a thread is terminated, this thread ID may be reused.

Some methods in this interface take a thread ID or an array of thread IDs as the input parameter and return per-thread information.

Thread CPU time

A Java virtual machine implementation may support measuring the CPU time for the current thread, for any thread, or for no threads.

The `isThreadCpuTimeSupported()` method can be used to determine if a Java virtual machine supports measuring of the CPU time for any thread. The `isCurrentThreadCpuTimeSupported()` method can be used to determine if a Java virtual machine supports measuring of the CPU time for the current thread. A Java virtual machine implementation that supports CPU time measurement for any thread will also support that for the current thread.

The CPU time provided by this interface has nanosecond precision but not necessarily nanosecond accuracy.

A Java virtual machine may disable CPU time measurement by default. The `isThreadCpuTimeEnabled()` and `setThreadCpuTimeEnabled(boolean)` methods can be used to test if CPU time measurement is enabled and to enable/disable this support respectively. Enabling thread CPU measurement could be expensive in some Java virtual machine implementations.

Thread Contention Monitoring

Some Java virtual machines may support thread contention monitoring. When thread contention monitoring is enabled, the accumulated elapsed time that the thread has blocked for synchronization or waited for notification will be collected and returned in the `ThreadInfo` object.

The `isThreadContentionMonitoringSupported()` method can be used to determine if a Java virtual machine supports thread contention monitoring. The thread contention monitoring is disabled by default. The `setThreadContentionMonitoringEnabled(boolean)` method can be used to enable thread contention monitoring.

Synchronization Information and Deadlock Detection

Some Java virtual machines may support monitoring of object monitor usage and ownable synchronizer usage. The `getThreadInfo(long[], boolean, boolean)` and `dumpAllThreads(boolean, boolean)` methods can be used to obtain the thread stack trace and synchronization information including which *lock* a thread is blocked to acquire or waiting on and which locks the thread currently owns.

The `ThreadMXBean` interface provides the `findMonitorDeadlockedThreads()` and `findDeadlockedThreads()` methods to find deadlocks in the running application.

Since:

1.5

See Also:

`ManagementFactory.getPlatformMXBeans(Class)`, `JMX Specification.`, `Ways to Access MXBeans`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`ThreadInfo[]`	`dumpAllThreads(boolean lockedMonitors, boolean lockedSynchronizers)`	Returns the thread info all live thre with stack trace and synchroniza information
`long[]`	`findDeadlockedThreads()`	Finds cycles threads tha are in deadl waiting to acquire obj monitors or **ownable synchroniz**
`long[]`	`findMonitorDeadlockedThreads()`	Finds cycles threads tha are in deadl waiting to acquire obj monitors.
`long[]`	`getAllThreadIds()`	Returns all thread IDs.
`long`	`getCurrentThreadCpuTime()`	Returns the total CPU ti for the curr thread in nanosecond
`long`	`getCurrentThreadUserTime()`	Returns the CPU time th the current thread has executed in user mode i nanosecond
`int`	`getDaemonThreadCount()`	Returns the current nun

		of live daem threads.
int	**getPeakThreadCount**()	Returns the peak live thread coun since the Ja virtual mac started or p was reset.
int	**getThreadCount**()	Returns the current nun of live threa including b daemon and non-daemon threads.
long	**getThreadCpuTime**(long id)	Returns the total CPU ti for a thread the specifie in nanosecond
ThreadInfo	**getThreadInfo**(long id)	Returns the thread info a thread of specified id with no stac trace.
ThreadInfo[]	**getThreadInfo**(long[] ids)	Returns the thread info each thread whose ID is the input ar ids with no stack trace.

ThreadInfo[]	getThreadInfo(long[] ids, boolean lockedMonitors, boolean lockedSynchronizers)	Returns the thread info each thread whose ID is the input ar ids, with st trace and synchroniza information
ThreadInfo[]	getThreadInfo(long[] ids, int maxDepth)	Returns the thread info each thread whose ID is the input ar ids, with st trace of a specified number of stack trace elements.
ThreadInfo	getThreadInfo(long id, int maxDepth)	Returns a thread info a thread of specified id with stack trace of a specified number of stack trace elements.
long	getThreadUserTime(long id)	Returns the CPU time th thread of th specified ID executed in user mode i nanosecond
long	getTotalStartedThreadCount()	Returns the total numbe threads cre and also started sinc the Java vir machine started.

boolean	**isCurrentThreadCpuTimeSupported**()	Tests if the virtual mac supports CP time measureme for the curr thread.
boolean	**isObjectMonitorUsageSupported**()	Tests if the virtual mac supports monitoring object moni usage.
boolean	**isSynchronizerUsageSupported**()	Tests if the virtual mac supports monitoring **ownable synchroniz** usage.
boolean	**isThreadContentionMonitoringEnabled**()	Tests if thre contention monitoring enabled.
boolean	**isThreadContentionMonitoringSupported**()	Tests if the virtual mac supports th contention monitoring.
boolean	**isThreadCpuTimeEnabled**()	Tests if thre CPU time measureme enabled.
boolean	**isThreadCpuTimeSupported**()	Tests if the virtual mac implementa supports CP time measureme for any thre
void	**resetPeakThreadCount**()	Resets the p thread coun

		the current number of l threads.
void	**setThreadContentionMonitoringEnabled**(boolean enable)	Enables or disables thr contention monitoring.
void	**setThreadCpuTimeEnabled**(boolean enable)	Enables or disables thr CPU time measureme

Methods inherited from interface java.lang.management.PlatformManagedObject

getObjectName

Method Detail

getThreadCount

```
int getThreadCount()
```

Returns the current number of live threads including both daemon and non-daemon threads.

Returns:

the current number of live threads.

getPeakThreadCount

```
int getPeakThreadCount()
```

Returns the peak live thread count since the Java virtual machine started or peak was reset.

Returns:

the peak live thread count.

getTotalStartedThreadCount

```
long getTotalStartedThreadCount()
```

Returns the total number of threads created and also started since the Java virtual machine started.

Returns:

`the total number of threads started.`

getDaemonThreadCount

```
int getDaemonThreadCount()
```

Returns the current number of live daemon threads.

Returns:

`the current number of live daemon threads.`

getAllThreadIds

```
long[] getAllThreadIds()
```

Returns all live thread IDs. Some threads included in the returned array may have been terminated when this method returns.

Returns:

`an array of long, each is a thread ID.`

Throws:

`SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").`

getThreadInfo

```
ThreadInfo getThreadInfo(long id)
```

Returns the thread info for a thread of the specified `id` with no stack trace. This method is equivalent to calling:

```
getThreadInfo(id, 0);
```

This method returns a `ThreadInfo` object representing the thread information for the thread of the specified ID. The stack trace, locked monitors, and locked synchronizers in the returned `ThreadInfo` object will be empty. If a thread of the given ID is not alive or does not exist, this method will return `null`. A thread is alive if it has been started and has not yet died.

MBeanServer access:
The mapped type of `ThreadInfo` is `CompositeData` with attributes as specified in the `ThreadInfo.from` method.

Parameters:

id - the thread ID of the thread. Must be positive.

Returns:

a ThreadInfo object for the thread of the given ID with no stack trace, no locked monitor and no synchronizer info; null if the thread of the given ID is not alive or it does not exist.

Throws:

IllegalArgumentException - if id <= 0.

SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").

getThreadInfo

ThreadInfo[] getThreadInfo(long[] ids)

Returns the thread info for each thread whose ID is in the input array ids with no stack trace. This method is equivalent to calling:

```
getThreadInfo(ids, 0);
```

This method returns an array of the ThreadInfo objects. The stack trace, locked monitors, and locked synchronizers in each ThreadInfo object will be empty. If a thread of a given ID is not alive or does not exist, the corresponding element in the returned array will contain null. A thread is alive if it has been started and has not yet died.

MBeanServer access:
The mapped type of ThreadInfo is CompositeData with attributes as specified in the ThreadInfo.from method.

Parameters:

ids - an array of thread IDs.

Returns:

an array of the ThreadInfo objects, each containing information about a thread whose ID is in the corresponding element of the input array of IDs with no stack trace, no locked monitor and no synchronizer info.

Throws:

IllegalArgumentException - if any element in the input array ids is <= 0.

SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").

getThreadInfo

```
ThreadInfo getThreadInfo(long id,
                         int maxDepth)
```

Returns a thread info for a thread of the specified `id`, with stack trace of a specified number of stack trace elements. The `maxDepth` parameter indicates the maximum number of `StackTraceElement` to be retrieved from the stack trace. If `maxDepth == Integer.MAX_VALUE`, the entire stack trace of the thread will be dumped. If `maxDepth == 0`, no stack trace of the thread will be dumped. This method does not obtain the locked monitors and locked synchronizers of the thread.

When the Java virtual machine has no stack trace information about a thread or `maxDepth == 0`, the stack trace in the `ThreadInfo` object will be an empty array of `StackTraceElement`.

If a thread of the given ID is not alive or does not exist, this method will return `null`. A thread is alive if it has been started and has not yet died.

MBeanServer access:
The mapped type of `ThreadInfo` is `CompositeData` with attributes as specified in the `ThreadInfo.from` method.

Parameters:

`id - the thread ID of the thread. Must be positive.`

`maxDepth - the maximum number of entries in the stack trace to be dumped. Integer.MAX_VALUE could be used to request the entire stack to be dumped.`

Returns:

`a ThreadInfo of the thread of the given ID with no locked monitor and synchronizer info. null if the thread of the given ID is not alive or it does not exist.`

Throws:

`IllegalArgumentException - if id <= 0.`

`IllegalArgumentException - if maxDepth is negative.`

`SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").`

getThreadInfo

```
ThreadInfo[] getThreadInfo(long[] ids,
                           int maxDepth)
```

Returns the thread info for each thread whose ID is in the input array `ids`, with stack trace of a specified number of stack trace elements. The `maxDepth` parameter indicates the maximum number of `StackTraceElement` to be retrieved from the stack trace. If `maxDepth == Integer.MAX_VALUE`, the entire stack trace of the thread will be dumped. If `maxDepth == 0`, no stack trace of the thread will be dumped. This method does not obtain the locked monitors and locked synchronizers of the threads.

When the Java virtual machine has no stack trace information about a thread or `maxDepth == 0`, the stack trace in the `ThreadInfo` object will be an empty array of `StackTraceElement`.

This method returns an array of the `ThreadInfo` objects, each is the thread information about the thread with the same index as in the `ids` array. If a thread of the given ID is not alive or does not exist, `null` will be set in the corresponding element in the returned array. A thread is alive if it has been started and has not yet died.

MBeanServer access:
The mapped type of `ThreadInfo` is `CompositeData` with attributes as specified in the `ThreadInfo.from` method.

Parameters:

`ids - an array of thread IDs`

`maxDepth - the maximum number of entries in the stack trace to be dumped. Integer.MAX_VALUE could be used to request the entire stack to be dumped.`

Returns:

`an array of the ThreadInfo objects, each containing information about a thread whose ID is in the corresponding element of the input array of IDs with no locked monitor and synchronizer info.`

Throws:

`IllegalArgumentException - if maxDepth is negative.`

`IllegalArgumentException - if any element in the input array ids is <= 0.`

`SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").`

isThreadContentionMonitoringSupported

`boolean isThreadContentionMonitoringSupported()`

Tests if the Java virtual machine supports thread contention monitoring.

Returns:

`true if the Java virtual machine supports thread contention monitoring; false otherwise.`

isThreadContentionMonitoringEnabled

`boolean isThreadContentionMonitoringEnabled()`

Tests if thread contention monitoring is enabled.

Returns:

`true if thread contention monitoring is enabled; false otherwise.`

Throws:

`UnsupportedOperationException - if the Java virtual machine does not support thread contention monitoring.`

See Also:

`isThreadContentionMonitoringSupported()`

setThreadContentionMonitoringEnabled

`void setThreadContentionMonitoringEnabled(boolean enable)`

Enables or disables thread contention monitoring. Thread contention monitoring is disabled by default.

Parameters:

`enable - true to enable; false to disable.`

Throws:

`UnsupportedOperationException - if the Java virtual machine does not support thread contention monitoring.`

`SecurityException - if a security manager exists and the caller does not have ManagementPermission("control").`

See Also:

`isThreadContentionMonitoringSupported()`

getCurrentThreadCpuTime

`long getCurrentThreadCpuTime()`

Returns the total CPU time for the current thread in nanoseconds. The returned value is of nanoseconds precision but not necessarily nanoseconds accuracy. If the implementation distinguishes between user mode time and system mode time, the returned CPU time is the amount of time that the current thread has executed in user mode or system mode.

This is a convenient method for local management use and is equivalent to calling:

```
getThreadCpuTime(Thread.currentThread().getId());
```

Returns:

the total CPU time for the current thread if CPU time measurement is enabled; -1 otherwise.

Throws:

`UnsupportedOperationException` - if the Java virtual machine does not support CPU time measurement for the current thread.

See Also:

`getCurrentThreadUserTime()`, `isCurrentThreadCpuTimeSupported()`, `isThreadCpuTimeEnabled()`, `setThreadCpuTimeEnabled(boolean)`

getCurrentThreadUserTime

`long getCurrentThreadUserTime()`

Returns the CPU time that the current thread has executed in user mode in nanoseconds. The returned value is of nanoseconds precision but not necessarily nanoseconds accuracy.

This is a convenient method for local management use and is equivalent to calling:

```
getThreadUserTime(Thread.currentThread().getId());
```

Returns:

the user-level CPU time for the current thread if CPU time measurement is enabled; -1 otherwise.

Throws:

`UnsupportedOperationException` - if the Java virtual machine does not support CPU time measurement for the current thread.

See Also:

`getCurrentThreadCpuTime()`, `isCurrentThreadCpuTimeSupported()`, `isThreadCpuTimeEnabled()`, `setThreadCpuTimeEnabled(boolean)`

getThreadCpuTime

```
long getThreadCpuTime(long id)
```

Returns the total CPU time for a thread of the specified ID in nanoseconds. The returned value is of nanoseconds precision but not necessarily nanoseconds accuracy. If the implementation distinguishes between user mode time and system mode time, the returned CPU time is the amount of time that the thread has executed in user mode or system mode.

If the thread of the specified ID is not alive or does not exist, this method returns `-1`. If CPU time measurement is disabled, this method returns `-1`. A thread is alive if it has been started and has not yet died.

If CPU time measurement is enabled after the thread has started, the Java virtual machine implementation may choose any time up to and including the time that the capability is enabled as the point where CPU time measurement starts.

Parameters:

`id - the thread ID of a thread`

Returns:

`the total CPU time for a thread of the specified ID if the thread of the specified ID exists, the thread is alive, and CPU time measurement is enabled; -1 otherwise.`

Throws:

`IllegalArgumentException - if id <= 0.`

`UnsupportedOperationException - if the Java virtual machine does not support CPU time measurement for other threads.`

See Also:

`getThreadUserTime(long)`, `isThreadCpuTimeSupported()`, `isThreadCpuTimeEnabled()`, `setThreadCpuTimeEnabled(boolean)`

getThreadUserTime

```
long getThreadUserTime(long id)
```

Returns the CPU time that a thread of the specified ID has executed in user mode in nanoseconds. The returned value is of nanoseconds precision but not necessarily nanoseconds accuracy.

If the thread of the specified ID is not alive or does not exist, this method returns `-1`. If CPU time measurement is disabled, this method returns `-1`. A thread is alive if it has been started and has not yet died.

If CPU time measurement is enabled after the thread has started, the Java virtual machine implementation may choose any time up to and including the time that the capability is enabled as the point where CPU time measurement starts.

Parameters:

`id - the thread ID of a thread`

Returns:

`the user-level CPU time for a thread of the specified ID if the thread of the specified ID exists, the thread is alive, and CPU time measurement is enabled; -1 otherwise.`

Throws:

`IllegalArgumentException - if id <= 0.`

`UnsupportedOperationException - if the Java virtual machine does not support CPU time measurement for other threads.`

See Also:

`getThreadCpuTime(long)`, `isThreadCpuTimeSupported()`, `isThreadCpuTimeEnabled()`, `setThreadCpuTimeEnabled(boolean)`

isThreadCpuTimeSupported

```
boolean isThreadCpuTimeSupported()
```

Tests if the Java virtual machine implementation supports CPU time measurement for any thread. A Java virtual machine implementation that supports CPU time measurement for any thread will also support CPU time measurement for the current thread.

Returns:

`true if the Java virtual machine supports CPU time measurement for any thread; false otherwise.`

isCurrentThreadCpuTimeSupported

```
boolean isCurrentThreadCpuTimeSupported()
```

Tests if the Java virtual machine supports CPU time measurement for the current thread. This method returns `true` if `isThreadCpuTimeSupported()` returns `true`.

Returns:

true if the Java virtual machine supports CPU time measurement for current thread; false otherwise.

isThreadCpuTimeEnabled

boolean isThreadCpuTimeEnabled()

Tests if thread CPU time measurement is enabled.

Returns:

true if thread CPU time measurement is enabled; false otherwise.

Throws:

UnsupportedOperationException - if the Java virtual machine does not support CPU time measurement for other threads nor for the current thread.

See Also:

isThreadCpuTimeSupported(), isCurrentThreadCpuTimeSupported()

setThreadCpuTimeEnabled

void setThreadCpuTimeEnabled(boolean enable)

Enables or disables thread CPU time measurement. The default is platform dependent.

Parameters:

enable - true to enable; false to disable.

Throws:

UnsupportedOperationException - if the Java virtual machine does not support CPU time measurement for any threads nor for the current thread.

SecurityException - if a security manager exists and the caller does not have ManagementPermission("control").

See Also:

isThreadCpuTimeSupported(), isCurrentThreadCpuTimeSupported()

findMonitorDeadlockedThreads

long[] findMonitorDeadlockedThreads()

Finds cycles of threads that are in deadlock waiting to acquire object monitors. That is, threads that are blocked waiting to enter a synchronization block or waiting to reenter a synchronization block after an Object.wait

call, where each thread owns one monitor while trying to obtain another monitor already held by another thread in a cycle.

More formally, a thread is *monitor deadlocked* if it is part of a cycle in the relation "is waiting for an object monitor owned by". In the simplest case, thread A is blocked waiting for a monitor owned by thread B, and thread B is blocked waiting for a monitor owned by thread A.

This method is designed for troubleshooting use, but not for synchronization control. It might be an expensive operation.

This method finds deadlocks involving only object monitors. To find deadlocks involving both object monitors and ownable synchronizers, the `findDeadlockedThreads` method should be used.

Returns:

`an array of IDs of the threads that are monitor deadlocked, if any; null otherwise.`

Throws:

`SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").`

See Also:

`findDeadlockedThreads()`

resetPeakThreadCount

```
void resetPeakThreadCount()
```

Resets the peak thread count to the current number of live threads.

Throws:

`SecurityException - if a security manager exists and the caller does not have ManagementPermission("control").`

See Also:

`getPeakThreadCount()`, `getThreadCount()`

findDeadlockedThreads

```
long[] findDeadlockedThreads()
```

Finds cycles of threads that are in deadlock waiting to acquire object monitors or ownable synchronizers. Threads are *deadlocked* in a cycle waiting for a lock of these two types if each thread owns one lock while trying to acquire another lock already held by another thread in the cycle.

This method is designed for troubleshooting use, but not for synchronization control. It might be an expensive operation.

Returns:

an array of IDs of the threads that are deadlocked waiting for object monitors or ownable synchronizers, if any; null otherwise.

Throws:

SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").

UnsupportedOperationException - if the Java virtual machine does not support monitoring of ownable synchronizer usage.

Since:

1.6

See Also:

isSynchronizerUsageSupported(), findMonitorDeadlockedThreads()

isObjectMonitorUsageSupported

boolean isObjectMonitorUsageSupported()

Tests if the Java virtual machine supports monitoring of object monitor usage.

Returns:

true if the Java virtual machine supports monitoring of object monitor usage; false otherwise.

Since:

1.6

See Also:

dumpAllThreads(boolean, boolean)

isSynchronizerUsageSupported

boolean isSynchronizerUsageSupported()

Tests if the Java virtual machine supports monitoring of ownable synchronizer usage.

Returns:

true if the Java virtual machine supports monitoring of ownable synchronizer usage; false otherwise.

Since:

1.6

See Also:

`dumpAllThreads(boolean, boolean)`

getThreadInfo

```
ThreadInfo[] getThreadInfo(long[] ids,
                           boolean lockedMonitors,
                           boolean lockedSynchronizers)
```

Returns the thread info for each thread whose ID is in the input array `ids`, with stack trace and synchronization information.

This method obtains a snapshot of the thread information for each thread including:

- the entire stack trace,
- the object monitors currently locked by the thread if `lockedMonitors` is `true`, and
- the ownable synchronizers currently locked by the thread if `lockedSynchronizers` is `true`.

This method returns an array of the `ThreadInfo` objects, each is the thread information about the thread with the same index as in the `ids` array. If a thread of the given ID is not alive or does not exist, `null` will be set in the corresponding element in the returned array. A thread is alive if it has been started and has not yet died.

If a thread does not lock any object monitor or `lockedMonitors` is `false`, the returned `ThreadInfo` object will have an empty `MonitorInfo` array. Similarly, if a thread does not lock any synchronizer or `lockedSynchronizers` is `false`, the returned `ThreadInfo` object will have an empty `LockInfo` array.

When both `lockedMonitors` and `lockedSynchronizers` parameters are `false`, it is equivalent to calling:

```
getThreadInfo(ids, Integer.MAX_VALUE)
```

This method is designed for troubleshooting use, but not for synchronization control. It might be an expensive operation.

MBeanServer access:
The mapped type of `ThreadInfo` is `CompositeData` with attributes as specified in the `ThreadInfo.from` method.

Parameters:

`ids - an array of thread IDs.`

`lockedMonitors - if true, retrieves all locked monitors.`

lockedSynchronizers - if true, retrieves all locked ownable synchronizers.

Returns:

an array of the ThreadInfo objects, each containing information about a thread whose ID is in the corresponding element of the input array of IDs.

Throws:

SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").

UnsupportedOperationException -

- if lockedMonitors is true but the Java virtual machine does not support monitoring of object monitor usage; or
- if lockedSynchronizers is true but the Java virtual machine does not support monitoring of ownable synchronizer usage.

Since:

1.6

See Also:

isObjectMonitorUsageSupported(), isSynchronizerUsageSupported()

dumpAllThreads

```
ThreadInfo[] dumpAllThreads(boolean lockedMonitors,
                            boolean lockedSynchronizers)
```

Returns the thread info for all live threads with stack trace and synchronization information. Some threads included in the returned array may have been terminated when this method returns.

This method returns an array of ThreadInfo objects as specified in the getThreadInfo(long[], boolean, boolean) method.

Parameters:

lockedMonitors - if true, dump all locked monitors.

lockedSynchronizers - if true, dump all locked ownable synchronizers.

Returns:

an array of ThreadInfo for all live threads.

Throws:

SecurityException - if a security manager exists and the caller does not have ManagementPermission("monitor").

UnsupportedOperationException -

- if lockedMonitors is true but the Java virtual machine does not support monitoring of object monitor usage; or

- if lockedSynchronizers is true but the Java virtual machine does not support monitoring of ownable synchronizer usage.

Since:

1.6

See Also:

isObjectMonitorUsageSupported(), isSynchronizerUsageSupported()

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.